Mahidol AI Community 2026 | AI Adoption Challenge with Kiro

# Kaelo: An AI-Assisted Paper Search Engine That Ranks by Reproducibility

Concept Paper • Based on the repository status at the end of the first prototype (October 2026)

Team Kaelo • Faculty of Information and Communication Technology (ICT)

**About the project.** This project is part of the **Mahidol AI Community 2026**, under the **AI Adoption Challenge with Kiro**. Kiro serves as both the development environment and the process framework used to build the system (see Section 3.4).

## 1 Motivation and Problem

Open Science [1] is a global movement that promotes the accessibility, transparency, and reusability of scientific research. It asks researchers to share not only their findings but also the data, code, and methods behind them, so that others can verify the results and build on them. This vision fits naturally with the daily work of researchers, because exploring the literature and conducting experiments both depend on being able to find, inspect, and reuse earlier work. As funders, journals, and research communities increasingly encourage open practices, the ability to reproduce published results is becoming a central measure of research quality.

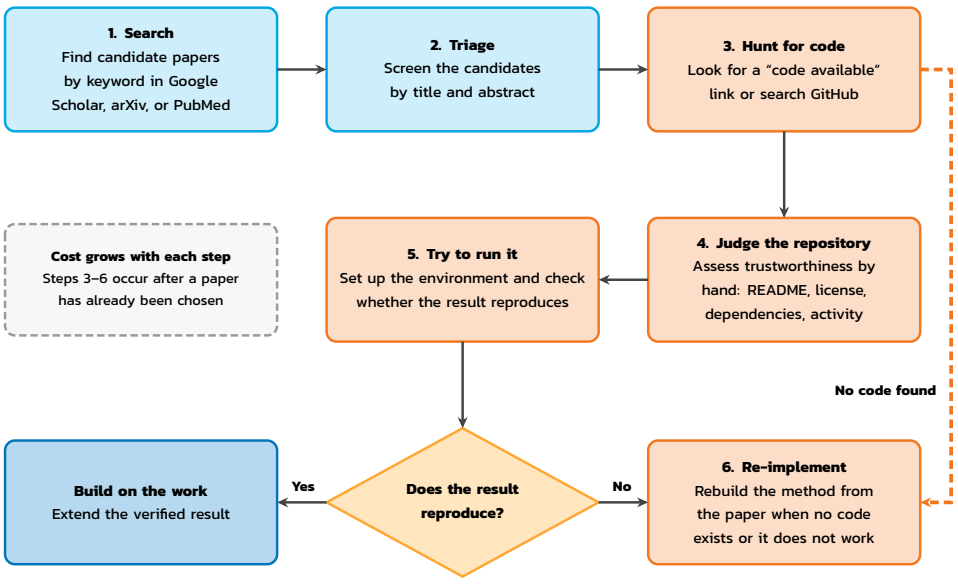



**Figure 1.** The current paper-search cycle compared with the Kaelo approach

In practice, however, paper search engines still overlook reuse, leaving it unobservable to researchers. A researcher who wants to build on someone else's work typically follows the same cycle. The researcher

[1] https://www.unesco.org/en/open-science

searches for papers, triages them by title and abstract, hunts for the accompanying code, judges the trustworthiness of the repository by hand, and tries to run it to see whether the result reproduces. If no code exists or it does not work, the researcher must re-implement the method from the paper.

The cost of this cycle is unevenly distributed. Steps 3 to 6 in Figure 1 are expensive and occur after a paper has been chosen, so the effort is spent on papers that may turn out to be unusable. Current search tools optimize only step 2, ranking results by textual relevance and citation counts, which say nothing about whether a result can be reproduced. As a result, the openness that the Open Science movement encourages is invisible at the moment researchers decide what to read. The key gaps that follow from this are summarized below.

**Key gaps**

- Search ranking ignores reproducibility, so researchers spend time on papers that cannot be reproduced.
- "Code available" is treated as a yes/no fact, so a link to an empty stub repository counts the same as a fully documented, versioned artifact.
- Papers and code drift apart (hyperparameters, data splits, metrics) and nobody notices until a reproduction attempt fails.
- Standards such as TRIPOD+AI, ACM Artifact Badging, the ML Code Completeness Checklist, and FAIR4RS are written for authors and reviewers and are not applied at search time. Authors lack a quick way to self-check, and when no code exists, readers must start from scratch.

> **Problem statement.** Researchers lack a fast, standards-grounded way to find papers that are likely to reproduce, to see why a paper received its score, or to close the gap when code is missing or inconsistent with the paper.

**Kaelo** (working title "Paper Search Engine supporting Open Science") addresses these gaps at the content level, without running any code. For each paper, it inspects the evidence that the paper and its repository make available, such as the README, license, dependency files, and the Experiment Setup reported in the paper. It then compares this evidence with published standards for the paper's field and measures how closely the code agrees with what the paper describes, such as hyperparameters, data splits, and metrics. Each criterion is scored with a confidence value, and fixed formulas combine the results into a Reproducibility Score that gives more weight to paper–code agreement (0.70) than to code quality (0.30). The score comes with a category breakdown, the coverage of criteria that could be assessed, and a discrepancy report, so users can see which requirements are met, which are missing, and where the paper and the code disagree.

This score estimates how likely a result is to be reproducible and how much it conform to reporting standards. By moving the assessment from step 5 to step 2, Kaelo supports researchers from step 1 through step 6. It improves **Search** with AI-assisted retrieval and adds the Reproducibility Score to **Triage**. It replaces manual effort in **Hunt for code** and **Judge the repository** with per-criterion evidence and a discrepancy report. It narrows **Try to run it** to papers that are likely to reproduce, and it can optionally support **Re-implement** by generating starter code or aligning existing code with the paper.

## 2 Objectives and Methodology

### 2.1 Objectives

1. **Search.** Provide AI-assisted search over Computer Science and Medicine papers, merging OpenAlex and arXiv results into a single de-duplicated list.
2. **Score.** Attach a transparent Reproducibility Score to each paper, computed against published standards selected by the paper's field, and blend it into the ranking.
3. **Explain.** Show per-criterion evidence and a category breakdown so that users can see *where* a paper is weak (for example, "Match the Paper" or "Install & Set Up").
4. **Close the gap.** When the user opts in, generate improvement recommendation for papers without a repository, or align existing code with the paper's Experiment Setup compiled with the standard policy (e.g. Computer Science, and Medical software)
5. **Support authors.** Let authors (Writers) upload their own manuscript and source code and receive the same scoring and code generation for their own work.

The primary users are Readers and Writers, which are modes of a single account . Funders, journals, laboratories, and standards bodies are secondary stakeholders. Non-goals are storing full third-party papers or repositories, executing generated code, acting as a general web search engine, and serving as a peer-review platform.

### 2.2 Scoring Method: the Standards-Conformance Score

The Reproducibility Score is rule-based and defined in `scoring_rule.md` (v1.2); Figure 2 gives an overview.

- Each field has a criteria table listing weights, tags, plain-language categories, and source references. Medicine uses TRIPOD+AI (26 weighted criteria). Computer Science uses ACM Artifact Review and Badging v1.1, the ML Code Completeness Checklist, and FAIR4RS v1.0, merged into 25 criteria (criteria shared across sources are combined and their weights summed). Unknown fields use a domain-neutral baseline.
- Each criterion is tagged **Q** (code or tooling quality) or **D** (paper–code discrepancy). Within a field, the Q weights sum to 0.30 and the D weights sum to 0.70.
- A criterion scores 1 (met), 0.5 (partial), 0 (unmet or contradicted), or NA (not applicable). NA criteria are dropped and the remaining weights are renormalized within their group.
- Confidence $c$ adjusts each criterion score $s$ according to Equation (1), where $\pi$ is the base rate (0.5 when no cohort data is available). The final score is given by Equation (2).
- The method also reports conformance per reference standard, coverage, and a discrepancy report listing every point where the paper contradicts the code.

$$\hat{s} = c \cdot s + (1 - c) \cdot \pi \tag{1}$$

$$S = 0.30\, S_Q + 0.70\, S_D \tag{2}$$

Weighting group D more heavily reflects the central idea of the project: that a repository merely *exists* matters less than whether the code *matches what the paper says*.

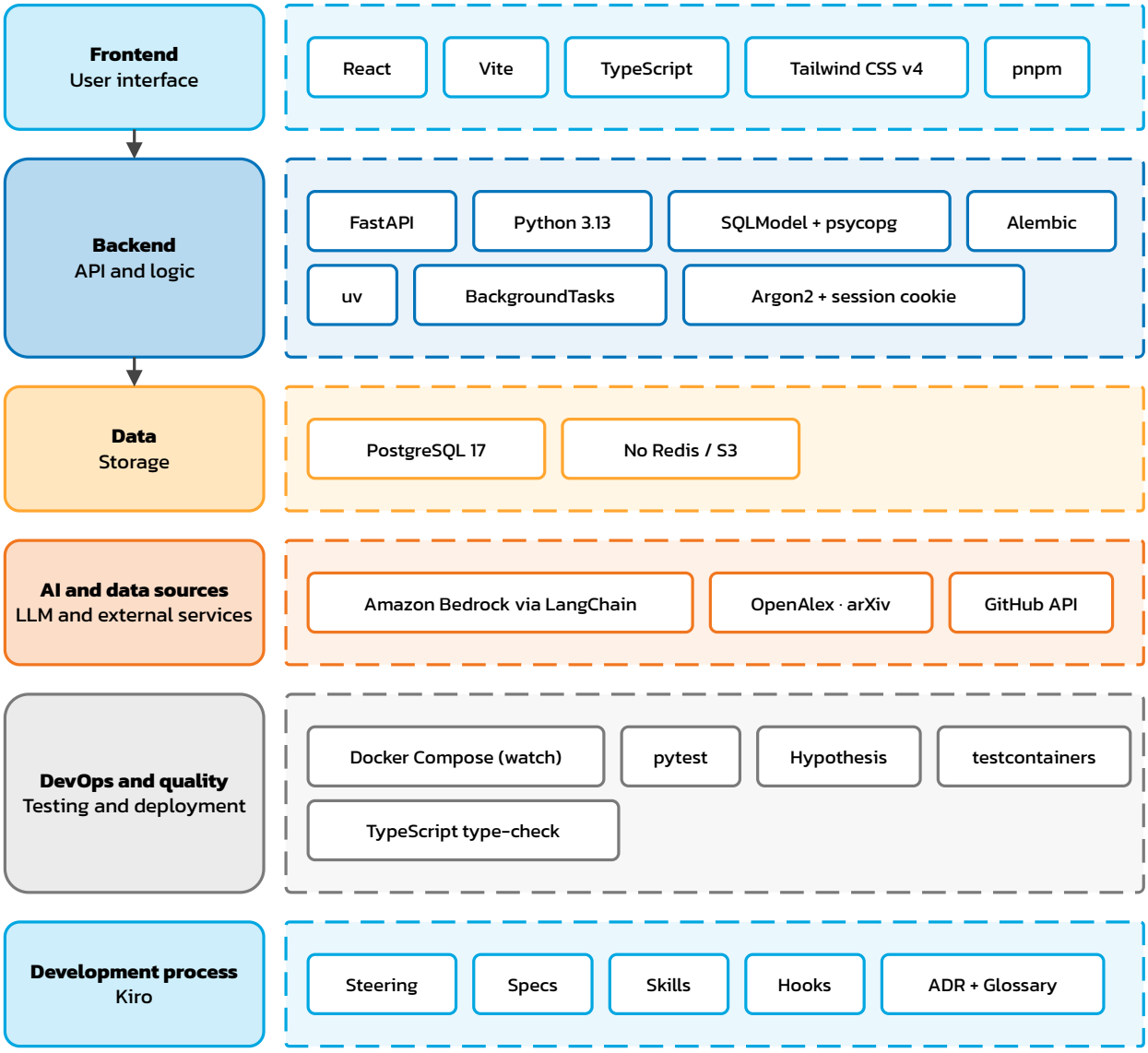



**Figure 2.** The scoring and ranking pipeline

### 2.3 Ranking, Data Policy, and Validation

The final ranking score is $0.3 \cdot \text{relevance} + 0.7 \cdot \text{reproducibility}$, with weights set in a configuration file. This blend is not produced by the LLM, and all component scores are shown to the user. Reader-side papers are stored as an index and links only; the sole exception is author-owned content that the user has consented to upload.

The validation plan, drawn from `scoring_rule.md` and mentor feedback, is as follows.

- Two independent human raters label about 100 papers per field, targeting Cohen's $\kappa \geq 0.6$ per criterion.
- Compare $S$, $S_Q$, and $S_D$ with external signals such as ACM Results badges and independent replication reports.
- Run adversarial tests (boilerplate README sections, empty config files, placeholder tests) to confirm that the score does not rise.
- Refit the weights from the data while keeping the group totals at 0.30/0.70. The mentor's suggestions are also being considered: verify that the content needed for reproduction actually exists, account for the original paper's environment and versions, treat security of generated code as a requirement, and add code-template generation.

## 3 System Design and Process

### 3.1 Architecture and Tech Stack

Figure 3 shows the system's tech stack layer by layer, from the frontend, backend, and data store to AI services, external data sources, testing tools, and the development process, and Table 1 summarizes the role of each layer. Everything runs in containers with Docker Compose.

**Table 1.** Tech stack and the role of each layer

| Layer | Technology | Role |
|---|---|---|
| Frontend | React, Vite, TypeScript, Tailwind CSS v4, pnpm | Reader and Writer UI; a single view-state machine with no router library |
| Backend | FastAPI, Python 3.13, SQLModel (synchronous psycopg), Alembic, uv | Routers and services (pure logic), with schema migrations |
| Background work | FastAPI `BackgroundTasks` | Scores are computed in the background (shown as `pending` first) and code generation runs as a cancellable job, with no broker |
| Authentication | Argon2, signed HTTP-only session cookie | Username and password sign-in |
| Data | PostgreSQL 17 | The only data store (no Redis or S3) |
| AI | Amazon Bedrock via LangChain | Called from the backend only; LangChain is the seam that allows model swapping |
| Data sources | OpenAlex, arXiv, GitHub API | Paper index and repository inspection |

| Layer | Technology | Role |
|---|---|---|
| DevOps and testing | Docker Compose (watch), pytest, Hypothesis, testcontainers, TypeScript type-check | The backend gate is `uv run pytest`; the frontend gate is the type-check |
| Development process | Kiro (steering, specs, skills, hooks) | See Section 3.4 |

The features are divided into five specs: `accounts` (registration, sign-in, Reader/Writer mode switching), `paper-search` (query expansion, merged search, result cards), `reproducibility-scoring` (standard selection, per-criterion evaluation, deterministic aggregation), `code-generation` (Generate when no repository exists, Align when one does), and `writer-workspace` (paper claims, PDF and source uploads).

## 3.2 Process Flows

**Reader flow**

1. The LLM expands the natural-language query. The system retrieves results from OpenAlex and arXiv, then merges, de-duplicates, and caches them as `Paper` rows, recording the field (domain) from the data source's own classifier.
2. The LLM writes a short relevance explanation for each result; the blended score is computed by a non-AI function.
3. In the background, the scorer selects a standard by the paper's domain, inspects the GitHub repository (README, license, tests, activity), and asks the LLM for per-criterion evidence.
4. Deterministic aggregation produces the score, and the result card updates from `pending` to its final value.
5. The detail page shows the score breakdown, the Experiment Setup, and the opt-in Generate or Align buttons.

**Writer flow.** The author claims a paper by DOI link or arXiv ID, uploads a PDF and source code (an archive or a public Git URL, text files only), and then runs scoring or code generation on the stored content.

## 3.3 Where the LLM Is and Is Not Used

**Table 2.** The role of the LLM and its guardrails

| Task | LLM role | Guardrail |
|---|---|---|
| Query expansion | Rewrites a natural-language query into search queries | The output is only an input to the search |
| Relevance and score explanations | Writes short explanations and turns the score breakdown into readable text | Display text only; it cannot change any number |
| Experiment Setup extraction | Extracts hyperparameters, datasets, and environment from the abstract, manuscript, or code into a structured record | Stored as bounded, structured data |

| Task | LLM role | Guardrail |
|---|---|---|
| Per-criterion evidence | Supplies status, confidence, and an evidence note for criteria that heuristics cannot decide | Treated as data, not control flow |
| Code generation and alignment | Generates code and a README from the Experiment Setup, manuscript, and source | Output is unverified and never executed; the Artifact Code-Quality Score is not fed back into the paper's score |

> **Key design rule: the LLM supplies evidence; the number is deterministic.** Tampering with LLM evidence can change only a criterion's status or note, while the weights, Q/D tags, and formulas are fixed code. The score is therefore traceable and resistant to prompt manipulation. Code generation uses delimiter-based output after a JSON-parsing approach proved brittle, and a mock mode (`CODEGEN_MOCK`) allows deterministic frontend testing without calling Bedrock.

### 3.4 Adoption of Kiro

This project is part of the Mahidol AI Community 2026 under the AI Adoption Challenge with Kiro. Kiro served as both the development environment and the process framework, and the repository records the workflow so that every contributor's agent follows the same cycle.

- **Steering** (`.kiro/steering/`). The files `product.md`, `tech.md`, `structure.md`, and `workflow.md` define product scope, technology constraints, repository structure, and the development cycle. A litmus test (will this still be true after the feature is done?) separates what belongs in steering from what belongs in a spec.
- **Specs** (`.kiro/specs/`). Five features were drafted as requirements, design, and tasks, then implemented one task at a time, with one PR per layer.
- **Skills and hooks.** Skills are pinned to a source commit and shared through git (`grill-with-docs`, `domain-modeling`, `ponytail`, `frontend-design`). A `UserPromptSubmit` hook nudges the agent to ask for a spec when a request looks like feature work; it is advisory only.
- **Documentation and verification.** Nine Architecture Decision Record(ADRs) and a glossary record decisions. The backend gate is `uv run pytest` (132 tests passing, using Hypothesis and testcontainers); the frontend gate is the type-check.

Observed effects: a shared vocabulary emerged (for example, the Code-Quality Score was split into Repository and Artifact variants), decisions survived changes in contributors, spec drift became visible, and parallel work produced few conflicts.

### 3.5 Current Status and Known Limitations

- All five specs have their core work implemented and were manually verified in the browser.
- Rewriting the scorer as the weighted Q/D model (the `standards/` adapters and schema migration) remains an open task; the passing scorer tests still cover the earlier blend. `/api/writer/*` has no dedicated tests yet.

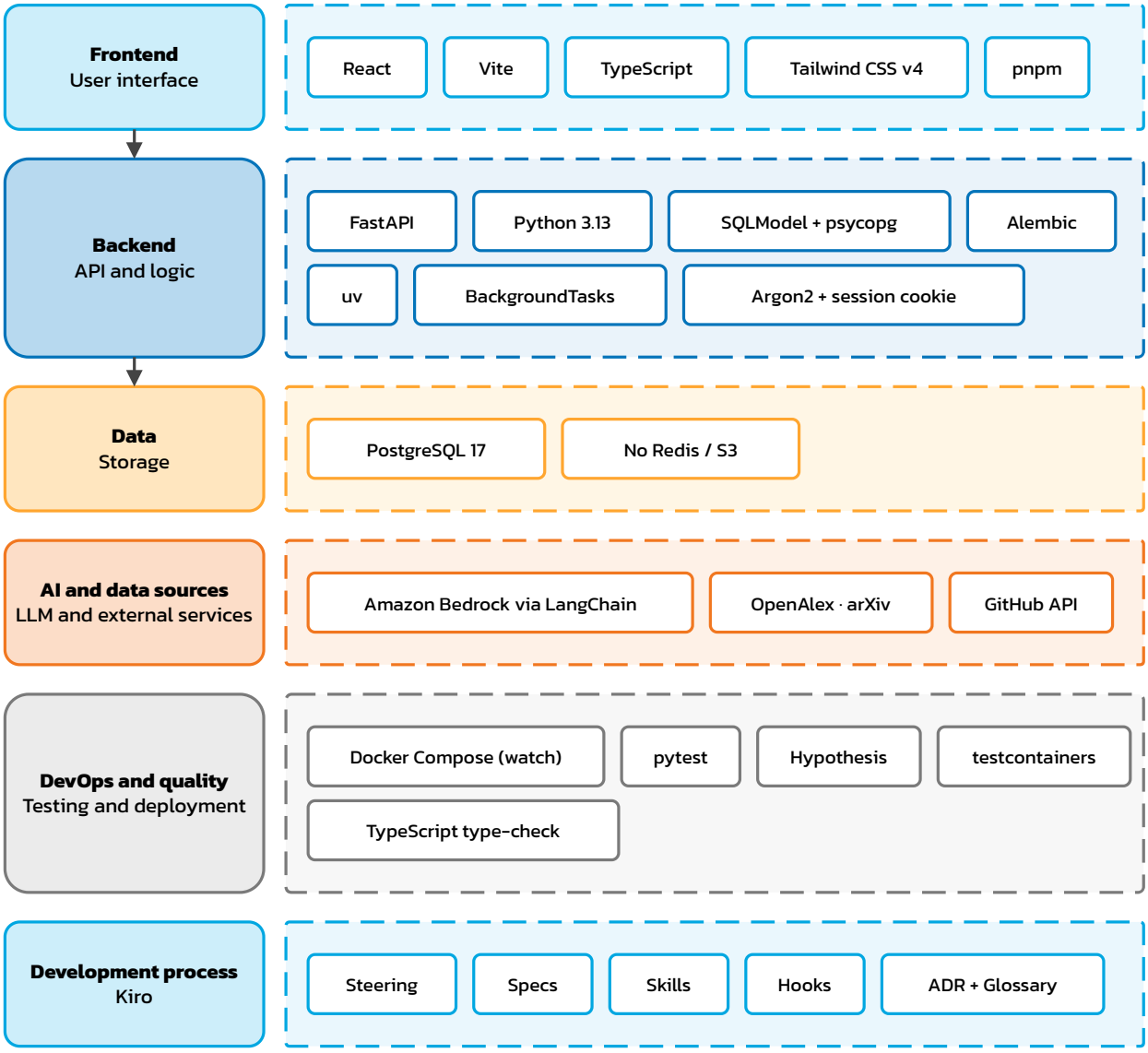



**Figure 3.** The Kaelo tech stack

- The first iteration inspects GitHub only, and Reader-side extraction uses only the title and abstract. Several FAIR4RS principles cannot be checked from a repository alone, so they carry low weight and confidence.
- Conformance to reporting standards is a proxy for reproducibility, not proof of it.

## 4 Future Work and Potential Deployment

### 4.1 Near Term: Completing the Prototype

1. Finish the v1.2 scoring design: `select_standard(field_of_study)` adapters for TRIPOD+AI, the merged CS standard, and the baseline, with the schema migration, and show the category breakdown and discrepancy report in the UI.
2. Replace the placeholder score explanations in the tooltip and add tests for `/api/writer/*`.
3. Harden generated code with policy checks (dependency and secret scanning) displayed beside the Artifact Code-Quality Score.
4. Add code-template generation (environment file, run script, config skeleton) as a lighter alternative to full generation.
5. Add a Review Report: a Markdown gap report derived from the existing Standard Result, with no second LLM call.

### 4.2 Research and Extensions

- **Research.** Conduct the 100-paper-per-field annotation study; fit the weights and the high/medium/low conformance thresholds against replication outcomes and ACM badge data; compare rankings with and without the reproducibility term in a user study (time to first successful run); and study the robustness of the LLM judge.
- **More sources and standards.** Add GitLab, Zenodo, and Hugging Face behind the existing `RepoHost` interface, Semantic Scholar and PubMed as sources, and standards such as PROBAST+AI, CONSORT, and biology or physics checklists, each as one adapter.
- **More features.** Resolve identifiers across types for Paper Claims (an arXiv URL and a DOI for the same paper currently produce two claims), present reproducibility as a calibrated probability, add opt-in sandboxed execution for the "Exercisable" criterion (`C-A5`), which would reverse a current non-goal and needs its own security review, and add a server-side session store for immediate sign-out revocation.
- **Deployment.** An institutional dashboard of reproducibility profiles, a pre-submission checker for journals and conferences (supporting, not replacing, artifact evaluation committees), a browser or reference-manager plugin that shows the score beside any paper, and a public API exposing the Standard Result.

### 4.3 Risks and Mitigations

This section identifies the main risks that could weaken Kaelo's reliability or its users' trust, and it pairs each risk with a mitigation that the design already includes or plans to add. Table 3 summarizes the result. The risks fall into three groups. The first group concerns the score itself. Score gaming, in which authors pad a README or add empty configuration files, and overstated compliance from the LLM are countered by adversarial tests, deterministic aggregation, confidence adjustment, and human spot checks. Because the weights currently reflect expert priors only, they will be refit from validation data before production use. The second group concerns the platform. Dependence on a single provider (Bedrock) is limited by the LangChain seam and a recorded architecture decision, and link rot and stale repositories are and led by periodic re-crawling and by recording the commit that was scored. The third group concerns generated code. Because it is unverified, it is available only on an opt-in basis, it carries a clear "unverified, not executed" disclaimer, and it never feeds back into the paper's score.

**Table 3.** Risks and mitigations

| Risk | Mitigation |
|---|---|
| Score gaming (padding the README, empty config files) | Adversarial tests, the D-heavy weighting, and human spot checks |
| The LLM overstates compliance | Deterministic aggregation, confidence adjustment, and sample agreement |
| Weights are only expert priors | Refit the weights from validation data before production use |
| Provider lock-in (Bedrock) | The LangChain seam and an ADR for this decision |
| Link rot and stale repositories | Periodic re-crawling and recording the commit that was scored |
| Unverified generated code misleads users | Opt-in only, a clear "unverified, not executed" disclaimer, and no feedback into the paper's score |

## References

ACM. Artifact Review and Badging, Version 1.1.

Chue Hong NP, et al. FAIR Principles for Research Software (FAIR4RS) v1.0. RDA, 2022. doi:10.15497/RDA00068.

Collins GS, et al. TRIPOD+AI statement. *BMJ* 2024;385:e078378.

Papers with Code. ML Code Completeness Checklist (releasing-research-code).

**Project documents**: `scoring_rule.md` v1.2, `GLOSSARY.md`, `docs/adr/0001–0009`, `.kiro/specs/*`, `summary-1st-prototype.md`